Technologies avancées du web

Architecture Monopage (SPA) / MEAN

-

Angular

Pierre Pompidor

Frameworks JavaScript côté client

Trois frameworks populaires

- **React** (Facebook) : plutôt une bibliothèque focalisée sur la rapidité d'exécution
- **Angular** (Google) : le plus complet mais le plus complexe (création de versions pour mobiles avec *Ionic*)
- **Vue.js** : mieux structuré que *React*, plus efficace qu'*Angular*

Frameworks JavaScript côté client

Trois frameworks populaires

- **React** (Facebook) : plutôt une bibliothèque focalisée sur la rapidité d'exécution
- **Angular** (Google) : le plus complet mais le plus complexe (création de versions pour mobiles avec *Ionic*)
- **Vue.js** : mieux structuré que *React*, plus efficace qu'*Angular*

Principales caractéristiques

- Composants développés à partir de classes **TypeScript**
- Extension du langage HTML (balises / composants statiques)
- Routage pour invoquer dynamiquement des composants
- Store pour partager des données entre composants

Frameworks JavaScript côté client

Trois frameworks populaires

- **React** (Facebook) : plutôt une bibliothèque focalisée sur la rapidité d'exécution
- **Angular** (Google) : le plus complet mais le plus complexe (création de versions pour mobiles avec *Ionic*)
- **Vue.js** : mieux structuré que *React*, plus efficace qu'*Angular*

Principales caractéristiques

- Composants développés à partir de classes **TypeScript**
- Extension du langage HTML (balises / composants statiques)
- Routage pour invoquer dynamiquement des composants
- Store pour partager des données entre composants

Deux modes : développement et production

Un gestionnaire de projet **ng**

Angular en mode développement

Un gestionnaire de projet **ng**

Un serveur de développement

- scrute les changements des codes
- transpile les codes ts en js avec *tsc*
- package les codes en un bundle avec *webpack*
- transmet ce bundle au navigateur

Angular en mode développement

Un gestionnaire de projet **ng**

Un serveur de développement

- scrute les changements des codes
- transpile les codes ts en js avec *tsc*
- package les codes en un bundle avec *webpack*
- transmet ce bundle au navigateur

Pour passer en production

il faut un **build** optimisé via le gestionnaire de projet :
ng build

Structure d'une application Angular

Structure minimal d'un projet (sans les fichiers de spécifications de tests)

```
projet
    package.json
    node_modules
    src
      \
          index.html
          styles.css
          assets
          app
           \
               app.component.ts
               app.component.html
               app.component.css
               app.routes.ts
               Composant1
                   \  ...
               Composant2
                   \  ...
```

Le composant "top-level"

index.html

```
<html><body> <app-root></app-root> </body></html>
```

Le composant "top-level"

index.html

```
<html><body> <app-root></app-root> </body></html>
```

app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
@Component({
  selector: 'app-root',
  standalone: true, imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent { title = 'projetDeTest'; }
```

Le composant "top-level"

index.html

```
<html><body> <app-root></app-root> </body></html>
```

app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
@Component({
  selector: 'app-root',
  standalone: true, imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent { title = 'projetDeTest'; }
```

app.component.html

```
<h1>Hello, {{ title }}</h1> <router-outlet />
```

Un second composant est géré par le composant top-level

second.component.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'second',
  templateUrl: './second.component.html',
})
export class SecondComponent {}
```

Un second composant est géré par le composant top-level

second.component.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'second',
  templateUrl: './second.component.html',
})
export class SecondComponent {}
```

Par l'insertion de son sélecteur dans le template du premier

```
<h1>Hello {{ title }}</h1> <app-second></app-second>
```

Un second composant est géré par le composant top-level

second.component.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'second',
  templateUrl: './second.component.html',
})
export class SecondComponent {}
```

Par l'insertion de son sélecteur dans le template du premier

```
<h1>Hello {{ title }}</h1> <app-second></app-second>
```

Via un routage vers → <router-outlet>

par manipulation de l'URL : http://localhost:4200/second
ou programmatiquement (exemple plus loin)

Affilier un composant statique

implémenté dans *second/second.component.ts*

```
import {Component} from '@angular/core';
import {RouterOutlet} from '@angular/router';
import {SecondComponent} from './second/second.component';

@Component({
  selector: 'app-root',
  standalone: true, imports:[RouterOutlet,SecondComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent { title = 'projetDeTest'; }
```

Affilier un composant statique

implémenté dans *second/second.component.ts*

```
import {Component} from '@angular/core';
import {RouterOutlet} from '@angular/router';
import {SecondComponent} from './second/second.component';

@Component({
  selector: 'app-root',
  standalone: true, imports:[RouterOutlet,SecondComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent { title = 'projetDeTest'; }
```

selecteur/directive *<app-second>* (→ *app.component.html*)

```
<h1>Hello {{ title }}</h1>
<app-second></app-second>
```

Le composant parent peut transmettre un input au composant enfant

```
<h1>Hello, {{ title }}</h1>
<app-second [message]="title"></app-second>
```

Le composant parent peut transmettre un input au composant enfant

```
<h1>Hello, {{ title }}</h1>
<app-second [message]="title"></app-second>
```

Les crochets indiquent que la string est évaluée comme du JS

Le composant parent peut transmettre un input au composant enfant

```
<h1>Hello, {{ title }}</h1>
<app-second [message]="title"></app-second>
```

Les crochets indiquent que la string est évaluée comme du JS

Pour les string, il est aussi possible d'utiliser l'interpolation

```
<app-second message="{{title}}"></app-second>
```

Le composant parent peut transmettre un input au composant enfant

```
<h1>Hello, {{ title }}</h1>
<app-second [message]="title"></app-second>
```

Les crochets indiquent que la string est évaluée comme du JS

Pour les string, il est aussi possible d'utiliser l'interpolation

```
<app-second message="{{title}}"></app-second>
```

Le composant enfant déclare l'input

```
export class SecondComponent {
    @Input() message: string = '';
}
```

Le routage

Les vues des composants routés sont insérés dans le DOM au niveau de l'élément correspondant à **router-outlet**

Le routage

Les vues des composants routés sont insérés dans le DOM au niveau de l'élément correspondant à **router-outlet**

app.component.html

```
<button (click)="invocation()"> composant dynamique </button>
<router-outlet />
```

Le routage

Les vues des composants routés sont insérés dans le DOM au niveau de l'élément correspondant à **router-outlet**

app.component.html

```
<button (click)="invocation()"> composant dynamique </button>
<router-outlet />
```

app.component.ts

```
export class AppComponent {
  constructor(private router: Router) {}
  invocation() { this.router.navigate(['second']); } }
```

Le routage

Les vues des composants routés sont insérés dans le DOM au niveau de l'élément correspondant à **router-outlet**

app.component.html

```
<button (click)="invocation()"> composant dynamique </button>
<router-outlet />
```

app.component.ts

```
export class AppComponent {
  constructor(private router: Router) {}
  invocation() { this.router.navigate(['second']); } }
```

Le routage peut se faire via <routerLink>

```
<button routerLink="/second"> Invocation </button>
<button [routerLink]="['/second','parametre']">...</button>
```

La table de routage doit être mise à jour

app.routes.ts (avant)

```
import { Routes } from '@angular/router';
export const routes: Routes = [];
```

La table de routage doit être mise à jour

app.routes.ts (avant)

```
import { Routes } from '@angular/router';
export const routes: Routes = [];
```

app.routes.ts (après)

```
import { Routes } from '@angular/router';
import { SecondComponent } from './second/second.component';
const routes:Routes=[{path:'second',component:SecondComponent
```

La table de routage doit être mise à jour

app.routes.ts (avant)

```
import { Routes } from '@angular/router';
export const routes: Routes = [];
```

app.routes.ts (après)

```
import { Routes } from '@angular/router';
import { SecondComponent } from './second/second.component';
const routes:Routes=[{path:'second',component:SecondComponent
```

Un composant n'est pas réinstancié s'il est déjà instancié

```
constructor(private router: Router){
  this.router.routeReuseStrategy.shouldReuseRoute=()=>false;
}
```

Les routes peuvent spécifier des paramètres

Une table de routage pour l'application d'ecommerce

```
export const routes: Routes = [
  {
      path: '',  // renvoie tous les articles
      component: ArticlesComponent,
      pathMatch: 'full'
  },
  {
      path: 'catalogue/:rayon/:marque/:prixMax',
      component: ArticlesComponent,
      pathMatch: 'full'
  },
  {
    path: '**', redirectTo: '', pathMatch: 'full'
  }
];
```

Création d'un projet Angular

Installation du client angular : **ng**

npm install @angular/cli

Création d'un projet Angular

Installation du client angular : **ng**

npm install @angular/cli

Création d'un projet

ng new <nom du projet>

Création d'un projet Angular

Installation du client angular : **ng**

npm install @angular/cli

Création d'un projet

ng new <nom du projet>

Création d'un composant / d'un service

ng generate component <nom> : ng g c <nom>
ng generate service <nom> : ng g s <nom>

Création d'un projet Angular

Installation du client angular : **ng**

npm install @angular/cli

Création d'un projet

ng new <nom du projet>

Création d'un composant / d'un service

ng generate component <nom> : ng g c <nom>
ng generate service <nom> : ng g s <nom>

Lancement du serveur de développement

ng serve -o
http://localhost:4200

Les services : partager un objet entre différents composants

- souvent liés à l'acquisition de données du serveur
- les méthodes d'acquisition renvoient des **observables**

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

@Injectable({providedIn: 'root'})
export class ProduitsService {
    private urlBase: string = 'http://localhost:8888/';
    constructor(private http: HttpClient) { }
    getProduits(): Observable<any> {
        return this.http.get(this.urlBase+'produits');
    }
}
```

Les services : utilisation par un composant

Injection de dépendance du singleton dans le constructeur et abonnement à l'observable

```
...
import { ProduitsService } from '../produits.service';
import { Observable } from 'rxjs';

@Component({...})
export class ProduitsComponent implements OnInit {
  public produits: any[] = new Array();
  constructor(private route: ActivatedRoute,
    private produitsService: ProduitsService) {}

  ngOnInit() {
   this.route.params.subscribe((params :Params) => {
    this.produitsService.getProduits().subscribe(produits=>{
        this.produits = produits;
      });
    }
  }
}
```

Les services : problématique de l'authentification

Création d'un observable

dont la valeur va pouvoir être modifiée dans le temps

Les services : problématique de l'authentification

Création d'un observable

dont la valeur va pouvoir être modifiée dans le temps

Authentification : authentification.service.ts

```
...
@Injectable ({providedIn: 'root'})
export class AuthService {
public user:Subject<string>=new BehaviorSubject<string>("");

  constructor(private http: HttpClient) {}
  getUser() {return this.user;}
  connect(data: string) {this.user.next(data);}
  disconnect() {this.user.next("");}

  verification(identifiants:any): Observable<any> {
    return this.http.post('http:.../user/connexion',
    JSON.stringify(identifiants), httpOptions);
  }
}
```

Les services : ... (connexion.component.ts/html)

```
import {AuthentificationService} from '../auth.service'
@Component({...}) export class ConnexionComponent {
  public user = {"email":"", "pass":""};
  constructor(private auth: AuthService, ...) { }
  onSubmit() {
    this.authService.verification(this.user)
        .subscribe(reponse => {
      if (reponse['resultat'])
        this.auth.connect(this.user.email);});}
}
```

Les services : ... (connexion.component.ts/html)

```
import {AuthentificationService} from '../auth.service'
@Component({...}) export class ConnexionComponent {
  public user = {"email":"", "pass":""};
  constructor(private auth: AuthService, ...) { }
  onSubmit() {
    this.authService.verification(this.user)
        .subscribe(reponse => {
      if (reponse['resultat'])
        this.auth.connect(this.user.email);});}
}
```

```
<form (ngSubmit)="onSubmit()" #inscriptionForm="ngForm">
  <label> Email </label>
  <input type="text" [(ngModel)]="user['email']" ... />
  <label> Mot de passe </label>
  <input type="password" [(ngModel)]="user['pass']" ... />
  <input type="submit" value="Se␣connecter"/>
```

Les services : problématique de l'authentification (suite)

Le composant qui liste les produits

utilise le service d'authentification

```
export class ProduitsComponent {
  public user: Observable<string>;
  public produits: any[] = new Array();
  constructor(private route: ActivatedRoute,
        private authService: AuthentificationService,
        private produitsService: ProduitsService) {
    this.user = this.authService.getUser();
  }...}
```

Les services : problématique de l'authentification (suite)

Le composant qui liste les produits

utilise le service d'authentification

```
export class ProduitsComponent {
  public user: Observable<string>;
  public produits: any[] = new Array();
  constructor(private route: ActivatedRoute,
        private authService: AuthentificationService,
        private produitsService: ProduitsService) {
    this.user = this.authService.getUser();
  }...}
```

```
@for (produit of produits; track produit.nom) {
  <div> {{produit.nom}} ({{produit.type}})
    <div *ngIf="(user|async)">
      <button> Achetez ! </button> </div>
  </div>
}
```

Les directives structurelles

for : ancienne version

```
<div *ngFor="let produit of produits">
  <div {{produit.nom}} ({{produit.type}}) </div>
</div>
```

Les directives structurelles

for : ancienne version

```
<div *ngFor="let produit of produits">
  <div {{produit.nom}} ({{produit.type}}) </div>
</div>
```

for : nouvelle version (inclut un tracker sur le DOM)

```
@for (produit of produits; track produit.nom) {
  <div {{produit.nom}} ({{produit.type}}) </div>
}
```

Les directives structurelles

for : ancienne version

```
<div *ngFor="let produit of produits">
  <div {{produit.nom}} ({{produit.type}}) </div>
</div>
```

for : nouvelle version (inclut un tracker sur le DOM)

```
@for (produit of produits; track produit.nom) {
  <div {{produit.nom}} ({{produit.type}}) </div>
}
```

Le tracker peut être avantageux pour un composant statique

- un itérable (ici *produits*) est partagé par plusieurs composants
- le composant réaffiche sa vue dès que l'itérable est modifié

Les directives structurelles (suite)

if : ancienne version

```
<div *ngIf="...">
    ...
</div>
```

Les directives structurelles (suite)

if : ancienne version

```
<div *ngIf="...">
    ...
</div>
```

if : nouvelle version

```
@if () {
    ...
}
@else {
    ...
}
```

Les formulaires et le one/two-way data binding

ngForm permet le one-way binding

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input type="text" name="login"
         ngModel required #log="ngModel">
  <input type="password" name="pass"
         ngModel required #pass="ngModel">
  <button type="submit"> Signin </button> </form>
<p> {{ log.value }} </p> <p> {{ pass.value }} </p>
```

Les formulaires et le one/two-way data binding

ngForm permet le one-way binding

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input type="text" name="login"
         ngModel required #log="ngModel">
  <input type="password" name="pass"
         ngModel required #pass="ngModel">
  <button type="submit"> Signin </button> </form>
<p> {{ log.value }} </p> <p> {{ pass.value }} </p>
```

```
export class LoginComponent {
  onSubmit(form: NgForm) {//form.value : {login:'...', ...}
}}
```

Les formulaires et le one/two-way data binding

ngForm permet le one-way binding

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input type="text" name="login"
         ngModel required #log="ngModel">
  <input type="password" name="pass"
         ngModel required #pass="ngModel">
  <button type="submit"> Signin </button> </form>
<p> {{ log.value }} </p> <p> {{ pass.value }} </p>
```

```
export class LoginComponent {
  onSubmit(form: NgForm) {//form.value : {login:'...', ...}
}}
```

[(ngModel)] liaison bidirectionnelle : login <=> this.login

```
<input type="text" [(ngModel)]="login" name="login">
```

Une autre façon de partager des données entre composants : les **stores**

Un store est un objet centralisé

- A chaque fois qu'il est modifié, un nouvel état (state) de cet objet est créé
- Il est possible de suivre l'évolution de l'état de cet objet
- Cet objet est injecté dans les composants qui en ont besoin

Une autre façon de partager des données entre composants : les **stores**

Un store est un objet centralisé

- A chaque fois qu'il est modifié, un nouvel état (state) de cet objet est créé
- Il est possible de suivre l'évolution de l'état de cet objet
- Cet objet est injecté dans les composants qui en ont besoin

Techniquement

- Comme pour les observables, il faut utiliser la bibliothèque **@ngrx**
- Mais la syntaxe est beaucoup plus complexe !

Exemple de la mise en oeuvre d'un store pour gérer un panier

Cet exemple n'a aucune utilité fonctionnelle

- Un composant permet de rajouter un produit dans le panier
- Un autre composant permet de lister le panier
- Tous les deux partagent le store

Exemple de la mise en oeuvre d'un store pour gérer un panier

Cet exemple n'a aucune utilité fonctionnelle

- Un composant permet de rajouter un produit dans le panier
- Un autre composant permet de lister le panier
- Tous les deux partagent le store

```
app.component.html => ajout d'un produit dans le panier
app.component.ts
panier.store.ts
affichePanier
  \
  affiche-panier.component.html => affichage/suppression du p
  affiche-panier.component.ts
```

Exemple de la mise en oeuvre d'un store : les vues

app.component.html

```
<h1>{{ title }}</h1>
<button (click)="ajouterProduit()">Ajouter un produit</button
<router-outlet />
```

Exemple de la mise en oeuvre d'un store : les vues

app.component.html

```
<h1>{{ title }}</h1>
<button (click)="ajouterProduit()">Ajouter un produit</button
<router-outlet />
```

affiche-panier.component.html

```
<ul>
@for (produit of panier.getProduits(); track $index) {
    <li> {{produit}} </li>
}
</ul>
@if (panier.getProduits().length > 0) {
  <button (click)="viderPanier()">Vider le panier</button>
}
```

Suite de l'exemple : le code du store

panier.store.ts

```
import { ... } from '@ngrx/signals';
type produitsState = {produits : string[];}
export const initialState: produitsState = {produits: []}

export const PanierStore = signalStore(
    { providedIn: 'root' },
    withState(initialState),
    withMethods((store) => ({
      ajouterProduit(): void {
        patchState(store, (state) => {
          const produits = state.produits;
          produits.push("produit"+(produits.length+1));
          return { produits };
        });
      },
      getProduits(): any[] { return store.produits();},
    }))
)
```

Suite de l'exemple : les codes des composants

app.component.ts

```
import { Component, inject } from '@angular/core';
...
@Component({ ...
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class AppComponent {
  panier = inject(PanierStore);
  ajouterProduit = () => this.panier.ajouterProduit();
}
```

affiche-panier.component.ts

```
import { Component, inject } from '@angular/core';
import { PanierStore } from '../caddie.store';
@Component({...})
export class CommandesComponent {
    panier = inject(PanierStore);
    viderPanier = () => this.panier.viderPanier();
}
```